Discrete Dynamic System with Hash Aggregation on Cyclic Slots

Zhandos Mambetali
zhandos.mambetali@gmail.com

April 5, 2026

MSC 2020: 37B15, 68Q80, 94A60

Abstract

We consider a discrete dynamic system of elements moving on a cyclic slot topology. Each element is characterized by a weight, speed, activation period, and initial position. For each slot, an aggregation function over the multiset of element values is introduced. A special case is cryptographic hash aggregation. The dynamics, periodicity, and informational properties of the system are formalized.

1 Introduction

Let $C = \{e_1, \ldots, e_n\}$ be a set of elements moving on slots

$$S = \mathbb{Z}_N = \{0, 1, \ldots, N-1\}$$

over discrete time $t \in \mathbb{Z}_{\geq 0}$.

Each element is defined as:

$$e = (w_e, v_e, k_e, t_0^e, p_0^e, f_e)$$

where:

- $w_e > 0$ — weight,
- $v_e \in \mathbb{Z}$ — speed,
- $k_e \in \mathbb{Z}_{>0}$ — update period,
- t_0^e — start time,
- $p_0^e \in S$ — initial position,
- $f_e \in \{0, 1\}$ — frozen flag.

2 Element Dynamics

$$p_e(t) = \begin{cases} p_0^e, & f_e = 1 \text{ or } t < t_0^e, \\ \left(p_0^e + v_e \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor\right) \bmod N, & \text{otherwise.} \end{cases} \tag{1}$$

This defines discrete dynamics on the group $\mathbb{Z}_N$.

3 Slot Aggregation

Let $\phi_e(t)$ denote the value of element e.

For slot s:

$$M_s(t) = \{\{\phi_e(t) \mid p_e(t) = s\}\}$$

is the multiset of values.

A canonical encoding is introduced:

$$\text{Encode}(M_s(t))$$

(e.g., sorting + serialization).

Then:

$$Y_s(t) = F_s(\text{Encode}(M_s(t)))$$

4 Hash Aggregation

If

$$F_s = \text{hash}_s$$

then:

$$Y_s(t) = \text{hash}_s(\text{Encode}(M_s(t)))$$

This defines a **time-distributed hash with moving keys**.

5 Periodicity

Theorem 1. *The period of an element is:*

$$T_e = k_e \cdot \frac{N}{\gcd(v_e, N)}$$

Proof. The motion is defined by linear dynamics modulo N. The period equals the order of v_e in $\mathbb{Z}_N$ taking discretization by k_e into account. □

The overall period:

$$T = \text{lcm}(T_1, \ldots, T_n)$$

6 Informational Properties

6.1 Information Loss

Let C be the system state.

Then:

$$H(C \mid Y(t)) > 0$$

i.e., aggregation leads to information loss.

6.2 Cryptographic Case

If cryptographic hashes are used:

- recovering C from Y is computationally hard,
- collisions are possible: different states yield the same $Y(t)$.

7 2D Generalization

Grid:

$$S = \mathbb{Z}_N \times \mathbb{Z}_M$$

Dynamics:

$$\begin{aligned} x_e(t) &= (x_0^e + v_e^x \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor) \bmod N \\ y_e(t) &= (y_0^e + v_e^y \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor) \bmod M \end{aligned} \tag{2}$$

Aggregation:

$$Y_{x,y}(t) = F_{x,y}(\text{Encode}(M_{x,y}(t)))$$

where:

$$M_{x,y}(t) = \{\{\phi_e(t) \mid (x_e, y_e) = (x, y)\}\}$$

8 Interpretation

The system is equivalent to:

- a cellular automaton with external agents,
- a dynamic hash system,
- a particle model on a torus.

9 Restricted Subsets and Collision Prohibition

9.1 Partitioning Elements into Sets

Let elements be partitioned into subsets:

$$C = \bigsqcup_{\alpha \in A} C_\alpha$$

where A is the set of labels (types), and C_α are elements of type α.

Each type α has a subset of allowed slots:

$$S_\alpha \subseteq S$$

9.2 Restricted Dynamics

For element $e \in C_\alpha$, its position is restricted:

$$p_e(t) \in S_\alpha \quad \forall t$$

Movement follows the cyclic order within S_α.

Let $S_\alpha = \{s_0, s_1, \ldots, s_{m-1}\}$ be an ordered list. Define local index:

$$i_e(t) \in \mathbb{Z}_m$$

Dynamics:

$$i_e(t) = \begin{cases} i_0^e, & t < t_0^e \text{ or } f_e = 1, \\ i_0^e + v_e \cdot \left\lfloor \frac{t - t_0^e}{k_e} \right\rfloor \mod m, & \text{otherwise}, \end{cases}$$

$$p_e(t) = S_\alpha[i_e(t)]$$

9.3 Global Shift

A global shift can be introduced:

$$p'_e(t) = (p_e(t) + g(t)) \mod N$$

where $g(t)$ is a shift function (e.g., $g(t) = t$).

9.4 Collision and Overlap Prohibition

Let elements in slot s be:

$$E_s(t) = \{e \mid p_e(t) = s\}$$

Constraint:

$$\forall e_i, e_j \in E_s(t): \quad \alpha_i \neq \alpha_j \Rightarrow \text{prohibited}$$

Elements from different sets cannot occupy the same slot.

9.5 Conflict Resolution

Define a conflict resolution operator:

$$\mathcal{R}(E_s(t)) \subseteq E_s(t)$$

Strategies:

- discard conflicting elements,
- priority by weight,
- random selection,
- keep only one set.

Then:

$$\tilde{E}_s(t) = \mathcal{R}(E_s(t))$$

and aggregation:

$$Y_s(t) = F_s(\text{Encode}(\{\{\phi_e(t) \mid e \in \tilde{E}_s(t)\}\}))$$

9.6 Properties

- the system becomes a **restricted dynamic system**,
- competition for resources (slots) arises,
- dynamics depend on conflict resolution policy,
- potential deadlocks or set domination may occur.

10 Examples and Applications

10.1 Distributed Systems

Interpret elements e as tasks or data keys, slots s as compute nodes, and sets C_α as task classes (e.g., services or priorities).

Allowed slots $S_\alpha \subseteq S$ define node affinity or shard placement.

Dynamics $p_e(t)$ describe **task migration**:

$$p_e(t+1) = (p_e(t) + v_e) \mod N$$

Global shift $g(t)$ models:

- load redistribution,
- rebalancing,
- hash-ring changes.

Conflicts model:

- node resource limits,
- isolation policies (multi-tenant systems),
- forbidden co-location.

Thus, the system is a **deterministic dynamic sharding model**.

10.2 Scheduling Theory

Elements are tasks, slots are time intervals or processors, k_e is task period, t_0^e is readiness time.

$$p_e(t)$$

specifies when/where a task executes.

Slot restrictions S_α define allowable processors, time windows, or compatibility constraints.

Overlap prohibition:

$$\forall e_i \neq e_j : \; p_{e_i}(t) = p_{e_j}(t) \Rightarrow \text{conflict}$$

corresponds to **collision-free scheduling**.

Conflict operator $\mathcal{R}$ defines scheduler strategy:

- priority (by w_e),
- round-robin,
- random,
- greedy.

Hence, the system is a **dynamic periodic-task scheduling model**.

10.3 Complex Dynamic Systems

System as a whole:

$$Y(t) = \mathcal{F}(P(t), X(t))$$

exhibits properties of complex systems:

- **self-organization**: slot distribution forms without centralized control,
- **nonlinearity**: with nonlinear F_s (e.g., hashes),
- **parameter sensitivity**: small changes in k_e, v_e, or S_α yield qualitatively different dynamics,
- **emergence**: global patterns arise from local rules.

In particular:

- linear $F_s \to$ **kinetic particle model**,
- hash aggregation $\to$ **entropy mixer**,
- with restrictions $\to$ **multi-agent interaction system**.

10.4 Relation to Known Models

The system generalizes:

- rotor-router models (deterministic motion),
- consistent hashing (key distribution),
- cellular automata (local state aggregation),
- multi-agent systems (agent interactions).

Key feature:

element motion and aggregation occur simultaneously over time.

11 Kinematic Analogy and Inertial Frames

Each element e is a discrete particle with mass $m_e = w_e$, position

$$\vec{r}_e(t) = (x_e(t), y_e(t))$$

and velocity

$$\vec{v}_e = (v_e^x, v_e^y)$$

Dynamics:

$$\vec{r}_e(t+1) = \vec{r}_e(t) + \vec{v}_e \mod (N, M)$$

11.1 Inertial Reference Frames

Consider a frame moving with velocity $\vec{u}$:

$$\vec{r}_e'(t) = \vec{r}_e(t) - \vec{u}t$$
$$\vec{v}_e' = \vec{v}_e - \vec{u}$$

The system is invariant under discrete analogs of Galilean transformations.

11.2 Velocity Addition and Local Momentum

For a cell (x, y) with elements, linear aggregation:

$$F_{x,y} = \sum$$

yields:

$$Y_{x,y}(t) = \sum_{e:\,(x_e,y_e)=(x,y)} m_e \vec{v}_e$$

Equivalent to **total momentum** in the cell.
Average velocity (local frame):

$$\vec{u}_{x,y}(t) = \frac{\sum m_e \vec{v}_e}{\sum m_e}$$

Switch to local frame:

$$\vec{v}_e' = \vec{v}_e - \vec{u}_{x,y}(t)$$
$$\sum m_e \vec{v}_e' = 0$$

corresponding to the center-of-mass frame.

11.3 Interpretation

- each cell (x, y) defines a **local inertial frame**,
- aggregated $Y_{x,y}(t)$ corresponds to momentum,
- normalized system $(\vec{u}_{x,y}) \to$ center-of-mass frame,
- model dynamics allow reference frame changes without altering motion structure.

11.4 Nonlinear Case

If $F_{x,y}$ is nonlinear (e.g., hash):

- total momentum information is lost,
- invariance under frame change breaks,
- system loses physical interpretation but gains cryptographic properties.

12 Applications

- streaming / sketching,
- distributed systems,
- pseudorandom generators,
- time-based hashing,
- self-organization models.

12.1 On Turing Completeness

In the current formalization, the system is not Turing-complete.

With a fixed number of elements C and slots S, the system state space is finite. Thus, dynamics $P(t)$ are periodic, and the system lacks unbounded memory required for universal computation.

However, the model can be extended to become Turing-complete by introducing one or more of:

- unbounded number of slots (expandable "tape"),
- mechanisms for changing internal element states $\phi_e(t)$,
- local interaction rules between neighboring slots.

With these extensions, the system can simulate known Turing-complete computation models (e.g., cellular automata) and becomes Turing-complete.

Thus, the proposed construction serves as a base dynamic model with natural extension to a universal computing system.

13 Conclusion

We propose a universal model of dynamic aggregation with moving elements. The system combines properties of dynamic systems, cellular automata, and cryptographic hashes, making it suitable for analyzing complex distributed processes.